3. Project Design Phase

Script-Controlled ACL: Restrict Record Access Based on Field Value

Solution Architecture

The design layers four ACLs on the **u_institution_details** table. Each ACL is a record-type ACL for one operation and requires one custom role. The READ ACL is *advanced*: it also carries a data condition and a script.

Operation	Required role	Data condition	Script
Read	bb1	Branch is EEE	Yes (advanced = true)
Create	bb2	None	No
Write	bb3	None	No
Delete	bb4	None	No

Role Design

- **bb1**: view records (EEE branch only).
- **bb2**: create new records (New button).
- **bb3**: edit existing records.
- **bb4**: delete records.
- Roles are cumulative: a user needs bb1 to see records at all, then bb2, bb3 and bb4 to add create, edit and delete.

Access Decision Flow (READ ACL)

1. User opens the Institution Details list.
2. ServiceNow checks the required role (bb1) and the data condition (Branch is EEE).
3. The ACL script runs: admin returns true, bb1 holder returns true, everyone else returns false.
4. Only records for which the ACL evaluates to true are displayed.

ACL Script Design

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }
    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1')) {
        return true;
    }
    // Deny access for all others
    return false;
})();
```

Test User Design

The **EEE User** (First name: EEE, Last name: User) holds all four roles and is used through impersonation. The admin account is used to confirm unrestricted access.

Design Decisions

- A data condition is used for the branch filter so that the record value is evaluated for each row.
- Separate roles per operation allow progressive verification, one ACL at a time.